P4143R0: Constant evaluation when?

Audience: CWG
S. Davis Herring <herring@lanl.gov>
Los Alamos National Laboratory
March 27, 2026

Introduction

This paper partially addresses US 33 (065) by clarifying what evaluations of a (putative) constant expression take place. It does not proceed further as would be necessary to support visible side effects during translation (*e.g.*, output as in P2758) because those do not exist in C++26.

Background

Expressions like that in `X<f()>` are described as "manifestly constant-evaluated" ([expr.const]/27), whose note claims that they are "evaluated even in an unevaluated operand", but there is only the implication of the defined term to justify that in general. Certain places refer to the values of constant expressions with various degrees of indirectness (*e.g.*, [dcl.array]/1, [dcl.fct.spec]/4, [dcl.align]/4, [temp.arg.nontype]/2–3, and [except.spec]/2).

On the other hand, [expr.const]/5.1 implies that there might be multiple evaluations of a variable initializer that is potentially a constant expression by referring to different contract evaluation semantics for "The initialization, when evaluated". The footnote in [expr.const]/27 specifically describes the interpretation with contract assertions ignored as "a trial evaluation".

In practice, compilers evaluate just once, merely remembering when a contract violation occurs and discarding it if the expression turns out to be non-constant for some other reason such that they "shouldn't" have encountered it. However, conceptually there are apparently further evaluations for some constant expressions: namely, those that initialize constant-initialized variables with automatic storage duration and those that implement constant destruction. These would need to be evaluated during program execution in order for the objects in question to have the correct lifetime; note that in

```
node* search_for_meaning(node *head) {
  constexpr int answer = 42;
  if(!head || head->data == answer) return head;
  return search_for_meaning(head->next);
}
```

the `answer` in each recursive call is a different object (as can be observed via their addresses) and thus needs a constructor call if of an appropriate type. Note also that such evaluations would still be constant evaluations, as indicated by `if consteval`. They would also be able to call immediate functions during execution, although the distinction there is merely that they could pass arguments that were not (syntactically) be constant expressions.

Wording

Relative to N5032.

[lex.phases]

Change phase 7 (in paragraph 1):

- […]

  [*Note*: Previously translated translation units can be preserved individually or in libraries. The separate translation units of a program communicate ([basic.link]) by (for example) calls to functions whose names have external or module linkage, manipulation of variables whose names have external or module linkage, or manipulation of data files. — *end note*]

  While the tokens constituting translation units are being analyzed and translated, required instantiations are performed.

  [*Note*: This can include instantiations which have been explicitly requested ([temp.explicit]). — *end note*]

  The contexts from which instantiations may be performed are determined by their respective points of instantiation ([temp.point]).

  [*Note*: Other requirements in this document can further constrain the context from which an instantiation can be performed. For example, a constexpr function template specialization might have a point of instantiation at the end of a translation unit, but its use in certain constant expressions could require that it be instantiated at an earlier point ([temp.inst]). — *end note*]

  ~~Each instantiation results in new program constructs.~~ The program is ill-formed if any instantiation fails.

  During the analysis and translation of tokens, ~~certain~~each manifestly constant-evaluated expression~~s~~ ~~are~~is evaluated ([expr.const]). The values of these expressions affect the types named and reflection values used in the program and thus the syntactic analysis. Their evaluation also can produce side effects that affect that analysis. Constructs appearing at a program point *P* are analyzed in a context where each side effect of evaluating an expression *E* as a full-expression is complete if and only if

  1. *E* is the expression corresponding to a *consteval-block-declaration* ([dcl.pre]), and
  2. either that *consteval-block-declaration* or the template definition from which it is instantiated is reachable from ([module.reach])
     1. *P*, or

2. the point immediately following the *class-specifier* of the outermost class for which *P* is in a complete-class context ([class.mem.general]).

[*Example*:

```
class S {
  class Incomplete;

  class Inner {
    void fn() {
      /* p1 */ Incomplete i;    // OK
    }
  }; /* p2 */ ;

  consteval {
    define_aggregate(^^Incomplete, {});
  }
}; /* p3 */ ;
```

Constructs at p_1 are analyzed in a context where the side effect of the call to `define_aggregate` is ~~evaluated~~complete because

1. ~~*E* is the expression corresponding to~~the call appears in a consteval block, and
2. p_1 is in a complete-class context of `S` and the consteval block is reachable from p_3.

— *end example*]

[expr.const]

Remove the note in paragraph 1:

- Certain contexts require expressions that satisfy additional requirements as detailed in this subclause; other contexts have different semantics depending on whether or not an expression satisfies these requirements. Expressions that satisfy these requirements, assuming that copy elision ([class.copy.elision]) is not performed, are called constant expressions.

  [*Note*: Certain ~~C~~constant expressions ~~can be~~are evaluated during translation ([lex.phases]). — *end note*]

Change paragraph 5:

- A variable `v` is *constant-initializable* if

  1. the full-expression of its initialization is a constant expression when interpreted as a *constant-expression* with all contract assertions using the ignore evaluation semantic ([basic.contract.eval]),

     [*Note*: ~~Within~~In the course of this ~~evaluation~~determination, `std::is_constant_evaluated()` ([meta.const.eval]) ~~returns~~has the value `true`. — *end note*]

[*Note*: ~~The~~Furthermore, if the initialization~~, when evaluated,~~ is manifestly constant-evaluated, its evaluation during translation can still evaluate contract assertions with other evaluation semantics, resulting in a diagnostic or ill-formed program if a contract violation occurs. — *end note*]

2. immediately after the initializing declaration of `v`, the object or reference `x` declared by `v` is constexpr-representable, and

3. if `x` has static or thread storage duration, `x` is constexpr-representable at the nearest point whose immediate scope is a namespace scope that follows the initializing declaration of `v`.

Change paragraph 27:

- An expression or conversion is *manifestly constant-evaluated* if it is:

1. a *constant-expression*, or

2. the condition of a constexpr if statement ([stmt.if]), or

3. the expression corresponding to a *consteval-block-declaration* ([dcl.pre]), or

4. an immediate invocation, or

5. the result of substitution into an atomic constraint expression to determine whether it is satisfied ([temp.constr.atomic]), or

6. the initializer of a variable that is usable in constant expressions or has constant initialization ([basic.start.static]). [*Footnote*: Testing this condition can involve a ~~trial~~notional evaluation of its initializer, with evaluations of contract assertions using the ignore evaluation semantic ([basic.contract.eval]), as described above. — *end footnote*] [*Example*:

   […]

   — *end example*]

[*Note*: Except for a *static_assert-message*, a manifestly constant-evaluated expression is evaluated even in an unevaluated operand ([term.unevaluated.operand]). — *end note*]